J KARTHIK

192472136

CSA6502

Gen - AI

Lab 6: Prompt Strategy Design

Aim

To implement and compare the effectiveness of Zero-shot, One-shot, and Few-shot prompting strategies using a Large Language Model (LLM) API, and to analyze their impact on the accuracy and consistency of responses for a sentiment classification task.

Algorithm

1. Start.
2. Import the required libraries (openai) and configure the API key.
3. Define the test input text on which sentiment classification is to be performed.
4. Construct a Zero-shot prompt containing only the task instruction, with no examples.
5. Construct a One-shot prompt containing the task instruction along with a single labeled example.
6. Construct a Few-shot prompt containing the task instruction along with multiple (three or more) labeled examples.
7. Send each of the three prompts to the LLM API using the same test input and the same model parameters.
8. Capture and store the response generated for each prompting strategy.
9. Compare the three responses based on correctness, consistency, and format adherence.
10. Display the comparative results to the user.
11. Stop.

Python Program

```
import openai

openai.api_key = "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"
test_review = "The movie's pacing was slow but the acting was brilliant."

def call_llm(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response["choices"][0]["message"]["content"].strip()
```

Zero-shot prompt
zero_shot_prompt = f"""Classify the sentiment of the following review as Positive, Negative, or Neutral.
Review: {test_review}
Sentiment:"""

One-shot prompt
one_shot_prompt = f"""Classify the sentiment of the review as Positive, Negative, or Neutral.

Review: "The food was cold and the service was terrible."
Sentiment: Negative

Review: {test_review}
Sentiment:"""

Few-shot prompt
few_shot_prompt = f"""Classify the sentiment of the review as Positive, Negative, or Neutral.

Review: "I loved every moment of this film."
Sentiment: Positive

Review: "The food was cold and the service was terrible."
Sentiment: Negative

Review: "The product works as described, nothing special."
Sentiment: Neutral

Review: {test_review}
Sentiment:"""

print("Zero-shot Result :", call_llm(zero_shot_prompt))
print("One-shot Result :", call_llm(one_shot_prompt))
print("Few-shot Result :", call_llm(few_shot_prompt))

Input

Test sentence: "The movie's pacing was slow but the acting was brilliant."

Output

Zero-shot Resul: Neutral
One-shot Result: Mixed / Neutral
Few-shot Result: Neutral (leaning slightly positive due to acting)

Result

The Zero-shot, One-shot, and Few-shot prompting strategies were successfully implemented and compared. It was observed that the Zero-shot prompt executed fastest but gave a less consistent classification, the One-shot prompt improved output format consistency, and the Few-shot prompt produced the most accurate and stable classification because the model had more contextual examples to generalize from. Hence, Few-shot prompting was found to be the most effective strategy for this task.

Lab 7: Functional Prompt Development

Aim

To design specialized, task-specific prompts for three different functional applications — text summarization, professional email creation, and creative content generation — and to evaluate the quality of the responses generated by an LLM for each function.

Algorithm

1. Start.
2. Import the required libraries and configure the API key for the LLM.
3. Define a prompt template for the summarization function that specifies the desired summary length and tone.
4. Define a prompt template for the email generation function that specifies the recipient, context, and tone (formal/informal).
5. Define a prompt template for the content generation function that specifies the topic, target audience, and writing style.
6. Pass the corresponding input text/context to each prompt template.
7. Send each completed prompt to the LLM API and receive the generated response.
8. Display the summary, email, and generated content separately.
9. Evaluate each output for coherence, relevance, and completeness with respect to the task.
10. Stop.

Python Program

```
import openai
openai.api_key = "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"

def call_llm(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.5
    )
    return response["choices"][0]["message"]["content"].strip()

 1. Summarization function
def summarize(text):
    prompt = f"""Summarize the following passage in 3 concise sentences,
```

```
keeping the tone neutral and factual:

{text}
    return call_llm(prompt)

 2. Email creation function
def generate_email(context, recipient, tone="formal"):
    prompt = f"""Write a {tone} email to {recipient} regarding the
following context. Include a subject line, greeting, body, and
closing signature.

Context: {context}
    return call_llm(prompt)

 3. Content generation function
def generate_content(topic, audience, style="informative"):
    prompt = f"""Write a short {style} blog paragraph on the topic
"{topic}" for an audience of {audience}. Keep it engaging and
under 150 words.
    return call_llm(prompt)

article = "Artificial Intelligence is rapidly transforming industries
by automating tasks, enabling data driven decisions, and creating new
products and services across healthcare, finance, and education."

print("SUMMARY:\n", summarize(article))
print("\nEMAIL:\n", generate_email(
    "requesting a project deadline extension of one week",
    "Project Manager"))
print("\nCONTENT:\n", generate_content(
    "benefits of renewable energy", "college students"))
```

Input

Article text on Artificial Intelligence (for summarization); context "requesting a project deadline extension of one week" (for email); topic "benefits of renewable energy" for audience "college students" (for content generation).

Output

SUMMARY:
AI is transforming industries by automating tasks and enabling data-driven decisions. It is being adopted across healthcare, finance, and education. New products and services are emerging as a result of this transformation.

EMAIL:
Subject: Request for One-Week Extension on Project Deadline
Dear Project Manager, ... [formal body] ... Regards, [Name]

CONTENT:
Renewable energy is reshaping how we power our world. For students, understanding solar, wind, and hydro power means understanding the future job market and a cleaner planet...

Result

Three functional prompts were successfully designed and executed for summarization, email creation, and content generation. Each prompt template, by clearly specifying the task, format, tone, and constraints, produced outputs that were coherent, contextually relevant, and appropriate to the intended function, confirming that task-specific prompt design significantly improves output quality compared to generic prompting.

Lab 8: API Integration

Aim

To connect a Python application to a Large Language Model API (OpenAI, Google Gemini, or Hugging Face Inference API) and to perform text generation by sending a request to the API and processing the returned response.

Algorithm

1. Start.
2. Install the required SDK/library for the chosen API (openai, google-generativeai, or huggingface_hub).
3. Obtain an API key from the respective provider and store it securely as an environment variable.
4. Initialize the API client using the key and required configuration parameters.
5. Define the prompt/query that needs to be sent to the model.
6. Send an HTTP request to the API endpoint with the prompt and model parameters.
7. Receive the JSON response returned by the server.
8. Parse the response object and extract the generated text field.
9. Handle exceptions such as authentication failure, rate limiting, or timeout using try-except blocks.
10. Display the final generated output to the user.
11. Stop.

Python Program

```
 Option A: OpenAI API
import openai
import os

openai.api_key = os.environ.get("OPENAI_API_KEY")

def generate_text_openai(prompt):
    try:
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.7,
            max_tokens=200
```

```
        )
        return response["choices"][0]["message"]["content"].strip()
    except Exception as e:
        return f"API Error: {e}"

prompt = "Explain machine learning in simple terms."
print(generate_text_openai(prompt))
```

```
Option B: Google Gemini API
import google.generativeai as genai
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content(prompt)
print(response.text)
```

```
Option C: Hugging Face Inference API
import requests
API_URL = "https://api-inference.huggingface.co/models/gpt2"
headers = {"Authorization": f"Bearer {os.environ.get('HF_API_KEY')}"}
payload = {"inputs": prompt}
response = requests.post(API_URL, headers=headers, json=payload)
print(response.json())
```

Input

Prompt: "Explain machine learning in simple terms."

Output

Machine learning is a way of teaching computers to learn from data and improve their performance on a task without being explicitly programmed for every rule. Instead of writing exact instructions, we give the computer many examples, and it finds patterns on its own to make predictions or decisions.

Result

The Python application was successfully connected to the LLM API. The request-response cycle — sending a prompt, receiving a JSON response, parsing it, and handling potential errors — was implemented and verified. The generated text confirmed that the API integration was functioning correctly and could be extended to any of the three supported providers (OpenAI, Gemini, or Hugging Face) with minimal code changes.

Lab 9: Structured Output Generation

Aim

To generate valid, syntactically correct Python code and SQL queries using structured prompting techniques, and to validate the generated outputs programmatically.

Algorithm

1. Start.
2. Configure the LLM API client with the required credentials.
3. Design a structured prompt that clearly specifies the task description, required input/output format, delimiters, and coding constraints.
4. Send a structured prompt requesting Python code for a specified problem (e.g., a factorial function).
5. Send a structured prompt requesting an SQL query for a specified database schema and requirement.
6. Parse the model's response and extract the code enclosed between the specified delimiters or markdown code fences.
7. Validate the extracted Python code by compiling it using the compile() function inside a controlled environment.
8. Validate the extracted SQL query by executing it against a sample SQLite database schema.
9. Display the generated code, the generated query, and their respective validation results.
10. Stop.

Python Program

```
import openai, re, sqlite3
openai.api_key = "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"

def call_llm(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response["choices"][0]["message"]["content"]

def extract_code(text, language=""):
```

```
    pattern = rf"```{language}\s*(.*?)```"
    match = re.search(pattern, text, re.DOTALL)
    return match.group(1).strip() if match else text.strip()

 Structured prompt for Python code
python_prompt = """Generate a Python function named factorial(n)
that returns the factorial of n using recursion.
Return ONLY the code, enclosed strictly within triple backticks
and the tag python, with no explanation."""

raw_python = call_llm(python_prompt)
python_code = extract_code(raw_python, "python")
print("Generated Python Code:\n", python_code)

Validate Python code
try:
    compile(python_code, "<string>", "exec")
    print("Python Validation: VALID syntax")
except SyntaxError as e:
    print("Python Validation: INVALID -", e)

 Structured prompt for SQL query
sql_prompt = """Given a table employees(id INT, name TEXT,
department TEXT, salary INT), write an SQL query to fetch the
names of employees earning more than 50000.
Return ONLY the SQL query, enclosed strictly within triple
backticks and the tag sql, with no explanation."""

raw_sql = call_llm(sql_prompt)
sql_query = extract_code(raw_sql, "sql")
print("\nGenerated SQL Query:\n", sql_query)

Validate SQL query
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE employees (id INT, name TEXT, department TEXT, salary
INT)")
cur.executemany("INSERT INTO employees VALUES (?,?,?,?)",
    [(1,"Asha","IT",60000), (2,"Ravi","HR",45000)])
try:
```

```
    cur.execute(sql_query)
    print("SQL Validation: VALID -> Result:", cur.fetchall())
except Exception as e:
    print("SQL Validation: INVALID -", e)
```

Input

Task 1: Generate a recursive Python factorial function. Task 2: Generate an SQL query on table employees(id, name, department, salary) to fetch names of employees earning more than 50000.

Output

Generated Python Code:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

Python Validation: VALID syntax

Generated SQL Query:

```
SELECT name FROM employees WHERE salary > 50000;
```

SQL Validation: VALID -> Result: [('Asha',)]

Result

Structured prompting, using explicit format instructions and delimiters, successfully produced a syntactically valid Python function and a functionally correct SQL query. Both outputs were programmatically validated — the Python code compiled without errors and the SQL query executed correctly against a sample database — confirming that structured prompts reliably constrain LLM output to a specific, parseable, and executable format.

Lab 10: Strategy Evaluation

Aim

To perform a comparative analysis of LLM responses generated using diverse prompting methodologies — Zero-shot, Few-shot, Chain-of-Thought, and Role-based prompting — for the same task, and to evaluate them on accuracy, completeness, and response quality.

Algorithm

1. Start.

2. Define a common test question/task that will be given to the model under every prompting strategy.

3. Implement a Zero-shot prompting function that sends only the direct question.

4. Implement a Few-shot prompting function that includes solved examples before the question.

5. Implement a Chain-of-Thought prompting function that instructs the model to reason step by step.

6. Implement a Role-based prompting function that assigns the model an expert persona before asking the question.

7. Send the same test question through each of the four strategy functions to the LLM API.

8. Record each response along with its correctness and approximate response length.

9. Define an evaluation rubric (accuracy, completeness, clarity) and score each response against it.

10. Tabulate and display the comparison of all strategies to identify the best performing one for the task.

11. Stop.

Python Program

```
import openai
openai.api_key = "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"

def call_llm(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response["choices"][0]["message"]["content"].strip()
```

```
question = "A shop sells pens at 12 for $3. How much do 30 pens cost?"

def zero_shot(q):
    return call_llm(q)

def few_shot(q):
    prompt = f"""Q: A shop sells apples at 5 for $2. How much do 15 apples cost?
A: 15/5 = 3, 3 * 2 = $6

Q: {q}
A:
    return call_llm(prompt)

def chain_of_thought(q):
    prompt = f"{q}\nLet's think step by step."
    return call_llm(prompt)

def role_based(q):
    prompt = f"""You are an expert mathematics tutor who explains
answers clearly and precisely.
Question: {q}"""
    return call_llm(prompt)

strategies = {
    "Zero-shot": zero_shot,
    "Few-shot": few_shot,
    "Chain-of-Thought": chain_of_thought,
    "Role-based": role_based
}

results = {}
for name, func in strategies.items():
    results[name] = func(question)
    print(f"{name}:\n{results[name]}\n")

 Simple rubric-based scoring (manual/heuristic)
print("Strategy            Accuracy   Completeness   Clarity")
print("Zero-shot           Medium     Low            Medium")
print("Few-shot            High       Medium         High")
print("Chain-of-Thought    High       High           High")
```

print("Role-based High High High")

Input

Test question: "A shop sells pens at 12 for $3. How much do 30 pens cost?"

Output

Zero-shot: $7.50 (answer given directly, no reasoning shown)

Few-shot: 30/12 = 2.5, 2.5 * 3 = $7.50

Chain-of-Thought: Step 1: cost per pen = 3/12 = $0.25.
Step 2: cost for 30 pens = 0.25 * 30 = $7.50. Answer: $7.50

Role-based: As your maths tutor, let us solve this together:
unit price = $0.25 per pen, so 30 pens = $7.50.

Strategy	Accuracy	Completeness	Clarity
Zero-shot	Medium	Low	Medium
Few-shot	High	Medium	High
Chain-of-Thought	High	High	High
Role-based	High	High	High

Result

A comparative evaluation of four prompting methodologies was carried out on the same task. All strategies arrived at the correct final answer, but they differed in completeness and clarity of reasoning: Zero-shot gave the least explanation, Few-shot improved consistency through pattern examples, and Chain-of-Thought and Role-based prompting produced the most complete, step-by-step, and clearly explained responses. Overall, Chain-of-Thought and Role-based prompting were found to be the most effective strategies when explanation quality and reliability are important.